

Hierarkier med exceptions

Många västrukturerade system definierar en egen klass för exceptions. Det är som tidigare nämnt helt tillåtet att kasta en klass som en exception. Vi skall definiera en enkel exception-klass:

```
#include <string>

// exception-klass
class Exception {
public:
    Exception (string Text = "Exception") : m_Text (Text) { }
    string m_Text;
};
```

Vi har här en enkel exception som innehåller en sträng som fungerar som ett meddelande. Detta meddelande kan t.ex. skrivas ut på skärmen eller skrivas till en loggfil. Texten som ges har det fördefinierade värdet "Exception". Ganska meningslöst, men fyller sin funktion. Notera att vi automatiskt initialiserar `m_Text` med den givna texten (se [avsnittet Initialisering av medlemmar i Kapitel 14](#)). Vi kan nu använda denna klass precis som vilken exception som helst.

Ärvda exceptions

Att ha en enkel klass för att representera en exception är kanske inte så väldigt praktiskt. Vi kan förstås skicka med en sträng som berättar vad som hänt, men det är svårt för ett `catch`-block att veta vad som gått fel. Det vore bra att kunna ha mera specificerade exceptions för att göra det lättare att skapa vettiga `catch`-block. Ingenting hindrar oss då ifrån att ärvda subclasser av vår `Exception`-klass:

```
#include <iostream>
#include <string>

// exception-klass
class Exception {
public:
    Exception (string Text = "Exception") : m_Text (Text) { }
    string m_Text;
};

// subclasser
class FileNotFound : public Exception {
public:
    FileNotFound (string Text = "File not found") : Exception (Text) { }
};

class PermissionDenied : public Exception {
public:
    PermissionDenied (string Text = "Permission denied") : Exception (Text) { }
};

class InvalidData : public Exception {
public:
    InvalidData (string Text = "Invalid data") : Exception (Text) { }
};
```

```
// funktion som genererar en exception
void test () {
    throw FileNotFound ( string("ingen sådan fil"));
}

int main () {
    // försök köra 'test'
    try {
        test ();
    }
    catch ( PermissionDenied E ){
        cout << "Kunde inte öppna filen: " << E.m_Text << endl;
    }
    catch ( FileNotFound E ){
        cout << "Hittade inte filen: " << E.m_Text << endl;
    }
    catch ( InvalidData E ){
        cout << "Felaktig data: " << E.m_Text << endl;
    }
    catch ( Exception E ){
        cout << "Exception: " << E.m_Text << endl;
    }
    catch ( ... ){
        cout << "Annan exception" << endl;
    }
}
```

Exemplet använder sig av tre subklasser till `Exception`. De alla kan ha ett unikt meddelande, och det är enkelt för `main()` att kontrollera vilken typ av exception det är frågan om. Notera att en exception i `catch`-block provas uppifrån och ned, så det första `catch`-block som passar kommer att användas, de övriga ignoreras. Så om vi t.ex. placerar det block som fångar `Exception E` först av alla kommer det att passa även alla subklasser av `Exception`, och därmed inte använda de specialiserade blocken längre ned. Som tumregel kan sägas att man bör placera de mest subklassade eller mest specialiserade blocken först, och gå mot basklasser längre ner. Har man . . . med bör den vara sist.

[Förutgående](#)

Ofångade exceptions

[Hem](#)[Upp](#)[Nästa](#)

Överlagring av operatorer